

Assignment/Report Title

Course Name: CSE 819, Robotics

Student Name: Gathik Jindal, Kanav Bhardwaj, Gautam Prasanna Kappagal

Student ID: IMT2023089, IMT2023024, IMT2023082

10 April 26

Abstract

This report details the environment, architecture, and configuration of the FedTraffic-RL project. The assignment implements a Deep Q-Network (DQN) agent designed to optimize traffic light timings at a four-way intersection. The simulation is run by the SUMO (Simulation of Urban MObility) engine. The agent is trained to maximize traffic throughput while minimizing local congestion and halting cars through adaptive phase switching.

1 Introduction

Traffic congestion is a highly dynamic problem requiring adaptive control systems. This project leverages a Reinforcement Learning (RL) approach to replace static, fixed-time traffic light controllers. The physical simulation operates within a predefined network featuring a 4-way intersection defined in `hello.net.xml`. The traffic demand—which includes cars, buses, and motorcycles with predefined routes—is loaded from `hello.rou.xml`.

2 Environment Setup

2.1 SUMO Simulation Environment

The simulation environment is built on SUMO and orchestrated via a Python interface using TraCI. The environment subjects the intersection to various deterministic traffic flows to ensure the agent learns robust policies:

- **Burst Spawn:** A high-volume surge of vehicles at the start of the simulation ($t = 0$) designed to test the agent's efficiency in clearing dense, sudden congestion.
- **Periodic Uniform:** A steady, regular flow of vehicles arriving at fixed intervals to test sustained throughput management over time.
- **Asymmetric Traffic:** A scenario (`one_heavy_three_light`) where one axis experiences heavy traffic periods while the others experience light traffic, demanding biased phase allocation.

The traffic signal system consists of 8 phases—four main Green phases governing North, East, South, and West traffic, alongside their respective 3-second Yellow transition phases. A mandatory minimum green time of 10 seconds is configured to prevent rapid, unsafe light switching.

2.2 State Representation and Action Space

To make intelligent decisions, the RL agent observes the intersection's status through an 18-dimensional continuous state vector:

- **Queues and Wait Times (8 dims):** The number of halting vehicles (normalized by 60) and accumulated waiting times (normalized by 2000) for each of the four incoming edges.
- **Phase Context (5 dims):** A one-hot encoded representation of the current active green phase and a normalized float representing the time elapsed in the current phase.

- **Traffic Pressure (4 dims):** The difference between the incoming vehicle count and the outgoing vehicle count for each axis, representing the immediate demand "pressure" on the intersection.
- **Bias (1 dim):** A constant bias term set to 1.0.

The **Action Space** is a discrete set of 4 actions. At each decision step, the agent outputs a choice representing which of the four directions (North, East, South, or West) should receive the next green light.

3 Agent and Training Setup

3.1 DQN Architecture

The core RL model is a Deep Q-Network. The neural network utilizes a standard feed-forward architecture with three fully connected linear layers. It ingests the 18-dimensional state vector, processes it through two hidden layers of 128 neurons each (activated by ReLU functions), and outputs Q-values corresponding to the 4 possible phase actions.

3.2 Reward Function

The behavior of the DQN agent is sculpted by a multi-objective reward formulation:

$$Reward = (W_1 \times \text{Flow Rate}) - (W_2 \times \text{Waiting Cars})$$

Where the Flow Rate represents vehicles successfully entering outgoing edges during a time step, and Waiting Cars serves as a penalty based on halting vehicles across all incoming lanes. The weights are configured to heavily prioritize throughput ($W_1 = 20.0$) while penalizing queues ($W_2 = 1.0$).

3.3 Training Configuration

The agent is trained using Experience Replay, utilizing a memory buffer of size 10,000 to randomly sample batches of size 64. A separate Target Network is maintained to stabilize the calculation of target Q-values. The model relies on Huber Loss and gradient clipping (max norm = 1.0) to ensure a stable and robust learning process, preventing exploding gradients. Optimization is handled by the Adam optimizer with a learning rate of 5×10^{-5} and a discount factor (γ) of 0.98. The agent explores its environment using an ϵ -greedy strategy, decaying ϵ from 1.0 down to a minimum of 0.01.

4 Conclusion

The integration of the environment and agent has yielded a highly stable and capable traffic control system. Single-agent training has shown consistent reward improvements and stabilized loss curves, successfully establishing a performance baseline. The modular nature of this setup successfully lays the groundwork for the next phase of the project, which will expand the architecture into a multi-intersection Federated Learning system using the Flower framework.